MAT102 — Applications of Systems of Linear Equations in Software Engineering

1 Introduction

For software engineers, Systems of Linear Equations (SLE) are not just abstract mathematical concepts; they are the foundation of algorithms that run the modern digital world. Whether you are load-balancing microservices, training machine learning models, or programming autonomous vehicles, you are constantly solving systems where multiple constraints must be met simultaneously

This session covers three practical domains where SLEs are essential:

1. Cloud Infrastructure & Load Balancing
2. Computer Vision & Image Filtering
3. Data Science & Polynomial Regression

2 Examples

Example 1: Cloud Infrastructure & Microservice Load Balancing

1.1 The Engineering Context:

A modern web application routes incoming API requests across multiple server clusters. To prevent any single cluster from crashing, the load balancer must distribute traffic according to the exact processing power of each cluster.

1.2 Problem Statement

An application receives exactly 1,000 requests per second (req/s). The load balancer distributes this traffic across three server clusters: A, B, and C.

- The total traffic must equal 1,000 req/s.
- Because Cluster A has newer hardware, it is configured to handle exactly twice the traffic of Cluster B.
- Cluster C acts as a buffer and is configured to process exactly 100 more req/s than Cluster B.

How much traffic should the load balancer route to each cluster?

1.3 Mathematical Formulation

Let x, y, and z represent the requests per second sent to clusters A, B, and C, respectively.

- Total traffic constraint: $x + y + z = 1000$
- Cluster A hardware constraint: $x = 2y \implies x - 2y + 0z = 0$
- Cluster C buffer constraint: $z = y + 100 \implies 0x - y + z = 100$

1.4 Gaussian Elimination Solution

Set up the augmented matrix:

$$\begin{bmatrix} 1 & 1 & 1 & | & 1000 \\ 1 & -2 & 0 & | & 0 \\ 0 & -1 & 1 & | & 100 \end{bmatrix}$$

Step 1: $R_2 \leftarrow R_2 - R_1$

$$\begin{bmatrix} 1 & 1 & 1 & | & 1000 \\ 0 & -3 & -1 & | & -1000 \\ 0 & -1 & 1 & | & 100 \end{bmatrix}$$

Step 2: Swap R_2 and R_3 to simplify pivot:

$$\begin{bmatrix} 1 & 1 & 1 & | & 1000 \\ 0 & -1 & 1 & | & 100 \\ 0 & -3 & -1 & | & -1000 \end{bmatrix}$$

Step 3: $R_3 \leftarrow R_3 - 3R_2$

$$\begin{bmatrix} 1 & 1 & 1 & | & 1000 \\ 0 & -1 & 1 & | & 100 \\ 0 & 0 & -4 & | & -1300 \end{bmatrix}$$

Back Substitution:

- From R_3: $-4z = -1300 \implies \mathbf{z = 325}$
- From R_2: $-y + 325 = 100 \implies \mathbf{y = 225}$
- From R_1: $x + 225 + 325 = 1000 \implies x + 550 = 1000 \implies \mathbf{x = 450}$

1.5 Software Engineering Interpretation

The routing algorithm must be programmed to send 450 req/s to Cluster A, 225 req/s to Cluster B, and 325 req/s to Cluster C. If traffic scales up to 10,000 req/s, the load balancer matrix is simply re-multiplied by a scalar of 10.

Example 2: Computer Vision & Sensor Calibration

2.1 The Engineering Context

In fields like autonomous robotics and 3D rendering, software must process pixel data to determine environmental factors (like lighting or depth). Image filters and feature extractors rely on finding specific "weights" to multiply against pixel values.

2.2 Problem Statement

An autonomous vehicle's perception system is analyzing a 3×3 grid of pixels. To normalize the lighting against sun glare, the software applies a color-correction filter with three distinct weights (w_1, w_2, w_3).Based on test data, applying these unknown weights to three different sample patches resulted in known output values:

- Patch 1: $1w_1 + 2w_2 + 3w_3 = 17$
- Patch 2: $2w_1 + 1w_2 + 1w_3 = 15$
- Patch 3: $3w_1 + 3w_2 + 1w_3 = 26$

Calculate the filter weights so they can be hardcoded into the perception algorithm.

2.3 Gaussian Elimination Solution

$$\begin{bmatrix} 1 & 2 & 3 & | & 17 \\ 2 & 1 & 1 & | & 15 \\ 3 & 3 & 1 & | & 26 \end{bmatrix}$$

Step 1: $R_2 \leftarrow R_2 - 2R_1$ and $R_3 \leftarrow R_3 - 3R_1$

$$\begin{bmatrix} 1 & 2 & 3 & | & 17 \\ 0 & -3 & -5 & | & -19 \\ 0 & -3 & -8 & | & -25 \end{bmatrix}$$

Step 2: $R_3 \leftarrow R_3 - R_2$

$$\begin{bmatrix} 1 & 2 & 3 & | & 17 \\ 0 & -3 & -5 & | & -19 \\ 0 & 0 & -3 & | & -6 \end{bmatrix}$$

Back Substitution:

- From R_3: $-3w_3 = -6 \implies \mathbf{w_3 = 2}$

- From R_2: $-3w_2 - 5(2) = -19 \implies -3w_2 = -9 \implies \mathbf{w_2 = 3}$
- From R_1: $w_1 + 2(3) + 3(2) = 17 \implies w_1 + 12 = 17 \implies \mathbf{w_1 = 5}$

2.4 Software Engineering Interpretation

The perception filter weights are $w_1 = 5$, $w_2 = 3$, and $w_3 = 2$. In a production environment, applying this matrix transformation allows the vision model to compensate for glare in real-time, effectively normalizing the video feed before passing it to the object-tracking neural network.

Example 3: Data Science & Predictive Modeling (Polynomial Fit)

2.1 The Engineering Context

Software engineers often need to predict how long an algorithm will take to run as the size of the data increases (Time Complexity). When analyzing an $O(n^2)$ algorithm, engineers use a system of linear equations to fit a parabolic curve to their benchmark data.

2.2 Problem Statement

You are profiling a new database sorting algorithm. You benchmarked the algorithm at three different input sizes (x, in millions of rows) and recorded the execution time (y, in seconds):

- 1 million rows took 4 seconds. $(1, 4)$
- 2 million rows took 9 seconds. $(2, 9)$
- 3 million rows took 18 seconds. $(3, 18)$

Assuming the time complexity follows a quadratic model $y = ax^2 + bx + c$, solve the system to find the coefficients a, b, and c.

2.3 Mathematical Formulation

Plug the (x, y) points into $y = ax^2 + bx + c$:

1. $a(1)^2 + b(1) + c = 4 \implies a + b + c = 4$
2. $a(2)^2 + b(2) + c = 9 \implies 4a + 2b + c = 9$
3. $a(3)^2 + b(3) + c = 18 \implies 9a + 3b + c = 18$

2.4 Gaussian Elimination Solution

$$\begin{bmatrix} 1 & 1 & 1 & | & 4 \\ 4 & 2 & 1 & | & 9 \\ 9 & 3 & 1 & | & 18 \end{bmatrix}$$

Step 1: $R_2 \leftarrow R_2 - 4R_1$ and $R_3 \leftarrow R_3 - 9R_1$

$$\begin{bmatrix} 1 & 1 & 1 & | & 4 \\ 0 & -2 & -3 & | & -7 \\ 0 & -6 & -8 & | & -18 \end{bmatrix}$$

Step 2: $R_3 \leftarrow R_3 - 3R_2$

$$\begin{bmatrix} 1 & 1 & 1 & | & 4 \\ 0 & -2 & -3 & | & -7 \\ 0 & 0 & 1 & | & 3 \end{bmatrix}$$

Back Substitution:

- From R_3: $\mathbf{c = 3}$
- From R_2: $-2b - 3(3) = -7 \implies -2b = 2 \implies \mathbf{b = -1}$
- From R_1: $a + (-1) + 3 = 4 \implies a + 2 = 4 \implies \mathbf{a = 2}$

2.5 Software Engineering Interpretation

The exact time complexity model for this algorithm is $y = 2x^2 - x + 3$. Using this model, the engineering team can accurately predict that processing a dataset of 10 million rows ($x = 10$) will take $2(100) - 10 + 3 =$ **193** seconds, allowing them to optimize timeout thresholds accordingly.

3 Exercises

Instructions for Students

Formulate a system of linear equations for each of the following real-world software engineering scenarios. Once formulated, use Gaussian Elimination (or substitution/elimination methods) to solve the system and clearly state your final engineering conclusion.

Exercise 1: Microservice Traffic Routing

Context: A backend system processes 1,200 requests per minute through an API gateway. The gateway routes traffic to three distinct microservices: Authentication (x), Database Query (y), and Image Processing (z).

Constraints:

- All 1,200 requests must be routed to one of the three services.
- The Authentication service (x) handles exactly twice as much traffic as the Database Query service (y).
- The Image Processing service (z) is configured to handle exactly 200 more requests than the Database Query service (y).

Task: Determine the exact number of requests per minute routed to each microservice.

Exercise 2: Cloud Container Allocation

Context: You are deploying a containerized web application using Docker. Your cloud server has exactly 32 CPU cores and 100 GB of RAM available. You need to deploy two types of containers:

- Web-UI Containers (x): Require 2 CPU cores and 4 GB of RAM each.
- Data-Processing Containers (y): Require 4 CPU cores and 14 GB of RAM each.

Task: How many of each container type should you deploy to utilize exactly 100% of the server's CPU and RAM without wasting any resources?

Exercise 3: Algorithm Time Complexity Modeling

Context: You are benchmarking a new sorting algorithm. You suspect its time complexity follows a quadratic curve represented by the equation $T = an^2 + bn + c$, where T is time in milliseconds and n is the input size in thousands of elements.

Benchmark Data:

- For $n = 1$, execution time $T = 2$ ms.
- For $n = 2$, execution time $T = 5$ ms.
- For $n = 3$, execution time $T = 10$ ms.

Task: Set up a system of three equations to find the exact values of the coefficients a, b, and c. What is the final time complexity equation?

Exercise 4: Computer Vision & Pixel Calibration

Context: In computer vision, normalizing an image against extreme sun glare requires calculating specific filter weights for the Red (R), Green (G), and Blue (B) pixel channels. You sample three reference pixels and apply the unknown weights (w_1, w_2, w_3) to achieve a target luminance.

Sample Data:

- Pixel 1: $1w_1 + 2w_2 + 1w_3 = 9$
- Pixel 2: $2w_1 + 1w_2 + 1w_3 = 8$
- Pixel 3: $1w_1 + 1w_2 + 2w_3 = 7$

Task: Solve the system to find the exact filter weights (w_1, w_2, w_3) the perception software must apply to the video feed.

Exercise 5: Autonomous Vehicle Sensor Fusion

Context: An autonomous vehicle relies on three sensors—LiDAR (L), Radar (R), and Camera (C)—to determine its distance from a physical obstacle. To prevent errors, the system assigns a trust weight to each sensor.

Constraints:

- The sum of all trust weights must equal exactly 1 (normalized): $L + R + C = 1$.
- In Test A, the weighted sum of the inputs was 3.3 meters: $5L + 2R + C = 3.3$.
- In Test B, the weighted sum of the inputs was 2.3 meters: $2L + 3R + 2C = 2.3$.

Task: Calculate the exact trust weights (L, R, C) hardcoded into the vehicle's localization algorithm.

Exercise 6: 3D Graphics Engine Lighting

Context: You are programming the rendering engine for a 3D simulation. A specific polygon is illuminated by three different directional light sources (L_1, L_2, L_3). The total light intensity hitting three distinct vertices of the polygon is measured:

- Vertex 1 receives light from all three sources equally:$L_1 + L_2 + L_3 = 15$.
- Vertex 2 is shielded from L_2, but receives double light from L_1: $2L_1 + 0L_2 + L_3 = 16$.
- Vertex 3 is shielded from L_1, but receives triple light from L_2: $0L_1 + 3L_2 + L_3 = 17$.

Task: Calculate the base intensity of each light source (L_1, L_2, L_3).

4 Instructor Answer Key

Exercise 1: Microservice Routing

- **System:**
  $x + y + z = 1200$
  $x - 2y = 0$
  $-y + z = 200$

- **Solution:** $x = 500$, $y = 250$, $z = 450$.

- **Conclusion:** Route 500 req to Auth, 250 to DB, and 450 to Image Processing.

Exercise 2: Cloud Container Allocation

- **System:**
  $2x + 4y = 32$ (CPU constraint)
  $4x + 14y = 100$ (RAM constraint)

- **Solution:** $x = 4$, $y = 6$.

- **Conclusion:** Deploy 4 Web-UI containers and 6 Data-Processing containers.

Exercise 3: Algorithm Time Complexity

- **System:**
  $a(1)^2 + b(1) + c = 2 \implies a + b + c = 2$
  $a(2)^2 + b(2) + c = 5 \implies 4a + 2b + c = 5$
  $a(3)^2 + b(3) + c = 10 \implies 9a + 3b + c = 10$

- **Solution:** $a = 1$, $b = 0$, $c = 1$.

- **Conclusion:** The exact complexity model is $T = n^2 + 1$.

Exercise 4: Computer Vision Calibration

- **System:**
  $w_1 + 2w_2 + w_3 = 9$
  $2w_1 + w_2 + w_3 = 8$
  $w_1 + w_2 + 2w_3 = 7$

- **Solution:** $w_1 = 2$, $w_2 = 3$, $w_3 = 1$.

- **Conclusion:** Apply weights 2, 3, and 1 to the RGB channels respectively.

Exercise 5: Autonomous Vehicle Sensor Fusion

- **System:**
  $L + R + C = 1$
  $5L + 2R + C = 3.3$
  $2L + 3R + 2C = 2.3$

- **Solution:** $L = 0.5$, $R = 0.3$, $C = 0.2$.

- **Conclusion:** Trust weights are 50% for LiDAR, 30% for Radar, and 20% for Camera.

Exercise 6: 3D Graphics Engine Lighting

- **System:**
  $L_1 + L_2 + L_3 = 15$
  $2L_1 + 0L_2 + L_3 = 16$
  $0L_1 + 3L_2 + L_3 = 17$

- **Solution:** $L_1 = 4$, $L_2 = 3$, $L_3 = 8$.

- **Conclusion:** Light intensities are 4, 3, and 8 units.

Prepared by Debasis Kumar